

2

Objektové vlastnosti

Triedy,
vlastnosti,
metódy, ...

Úloha 1

- **Stiahnite si .ipynb súbor**
- **Otvorte ho vo Visual Studio Code**
 - Aby ste mohli súbor .ipynb upravovať, musíte mať nainštalované rozšírenie **Polyglot Notebook** – ak ho nemáte, nainštalujte si ho
- **Realizujte jednotlivé úlohy podľa zadania**

Úloha 2 – Hudobná knižnica (1)

- Navrhnite a naprogramujte malý model „hudobnej knižnice“ s dôrazom na moderný C# (recordy, with, init, rozhrania, explicitná implementácia, enum, tuple, extension metódy)

Úloha 2 – Hudobná knižnica (2)

- Vytvorte projekt konzolovej aplikácie, v ktorej vytvoríte nasledujúce typy: **Genre**, **Song**, **PlaylistItem** a **DurationInfo**
 - PlaylistItem nech je record struct (hodnotový typ).
 - DurationInfo nech je readonly struct.
 - Cieľ: jednoduchý, nemenný dátový model vhodný na filtrovanie a agregáciu.

```
public enum Genre { Pop, Rock, Jazz, Classical, Electronic, Other }

public record Song(string Title, string Artist, Genre Genre, TimeSpan Duration);

public readonly record struct PlaylistItem(Song Song, int Order);

public readonly struct DurationInfo
{
    public TimeSpan Total { get; }
    public int Count { get; }
    public DurationInfo(TimeSpan total, int count) { Total = total; Count = count; }
    public double AverageSeconds() => Count == 0 ? 0 : Total.TotalSeconds / Count;
}
```

Úloha 2 – Hudobná knižnica (3)

- Definujte rozhrania pre prehrávanie a filtrovanie
 - Návrh: IPlayable má metódu Play();
 - IFilter má Filter(IEnumerable<Song>)

```
public interface IPlayable { void Play(); }

public interface IFilter
{
    IEnumerable<Song> Filter(IEnumerable<Song> source);
}
```

Úloha 2 – Hudobná knižnica (4)

- Vytvorte dve implementácie IFilter: GenreFilter a ArtistFilter.
 - GenreFilter: vyberie skladby podľa žánru.
 - ArtistFilter: case-insensitive porovnanie interpreta.

```
public class GenreFilter : IFilter
{
    private readonly Genre _genre;
    public GenreFilter(Genre genre) => _genre = genre;
    public IEnumerable<Song> Filter(IEnumerable<Song> source)
    {
        var result = new List<Song>();
        foreach (var s in source)
            if (s.Genre == _genre)
                result.Add(s);
        return result;
    }
}

public class ArtistFilter : IFilter
{
}
```

Úloha 2 – Hudobná knižnica (5)

- Trieda Playlist ukladá PlaylistItem a kontroluje poradie
 - Verejné Play(): detailný výpis.
 - Explicitné IPlayable.Play(): stručný výpis (iný režim).
 - Implementuje IEnumerable<PlaylistItem>

```
public class Playlist : IPlayable, IEnumerable<PlaylistItem>
{
    private readonly List<PlaylistItem> _items = new();

    public void Add(PlaylistItem item)
    {
        if (item.Order < 1) throw new ArgumentOutOfRangeException(nameof(item.Order));
        if (_items.Any(i => i.Order == item.Order)) throw new ArgumentException("Duplicate order.");
        _items.Add(item);
    }

    public void Play()
    {
        Console.WriteLine("--- PLAYLIST (detail) ---");
        var sorted = new List<PlaylistItem>(_items);
        sorted.Sort((a, b) => a.Order.CompareTo(b.Order));

        foreach (var i in sorted)
        {
            Console.WriteLine($"{i.Order}. {i.Song.Title} – {i.Song.Artist} [{i.Song.Genre}] {i.Song.Duration}");
        }
    }
}
```

Úloha 2 – Hudobná knižnica (6)

- Vytvorte PlaylistUtils.GetDurationInfo(IEnumerable<PlaylistItem>) – spočíta Total a Count.
 - Vytvorte Helpers.GetTopByDuration(IEnumerable<Song>, int) → vráti (TopByDuration, Summary).
 - TopByDuration: zoradiť skladby podľa dĺžky, zobrať prvé N. Summary: DurationInfo.

```
public static class PlaylistUtils
{
    public static DurationInfo GetDurationInfo(IEnumerable<PlaylistItem> items)
    {
        var total = TimeSpan.Zero; int count = 0;
        foreach (var i in items) { total += i.Song.Duration; count++; }
        return new DurationInfo(total, count);
    }
}

public static class Helpers
{
    public static (IEnumerable<Song> TopByDuration, DurationInfo Summary)
        GetTopByDuration(IEnumerable<Song> songs, int take)
    {
        var top = songs.OrderByDescending(s => s.Duration).Take(take).ToList();
        var total = top.Aggregate(TimeSpan.Zero, (acc, s) => acc + s.Duration);
        return (top, new DurationInfo(total, top.Count));
    }
}
```


Úloha 2 – Hudobná knižnica (7)

- Vytvorte Seed.Songs s reprezentatívnymi skladbami (rôzne žánre, dĺžky).
 - V Program.cs pripravte demo: filtre → playlist → agregácie → tuple → with → switch expression.

```
using static System.TimeSpan;
public static class Seed
{
    public static IReadOnlyList<Song> Songs => new List<Song>
    {
        new("Yellow Submarine", "The Beatles", Genre.Rock, FromMinutes(2)+FromSeconds(38)),
        new("Nocturne in E-flat", "Chopin", Genre.Classical, FromMinutes(4)+FromSeconds(10)),
        new("Take Five", "Dave Brubeck", Genre.Jazz, FromMinutes(5)+FromSeconds(24)),
        new("Halo", "Beyoncé", Genre.Pop, FromMinutes(3)+FromSeconds(44)),
        new("Aerodynamic", "Daft Punk", Genre.Electronic, FromMinutes(3)+FromSeconds(27))
    };
}
```

Úloha 2 – Hudobná knižnica (8)

- 1) Aplikujte filtre (Genre, Artist).
 - 2) Naplňte playlist bez duplicít poradia.
 - 3) Zobrazte sumár: Total, Count, AverageSeconds().
 - 4) Zavolajte Play() vs ((IPlayable)playlist).Play().
 - 5) Vyžiadaajte TopByDuration a Summary (tuple).

```
var all = Seed.Songs;
IFilter byGenre = new GenreFilter(Genre.Classical);
var classics = byGenre.Filter(all).ToList();
IFilter byArtist = new ArtistFilter("Coldplay");
var coldplay = byArtist.Filter(all).ToList();

var playlist = new Playlist();
int order = 1;
foreach (var s in classics.Concat(coldplay).Distinct())
    playlist.Add(new PlaylistItem(s, order++));

var info = PlaylistUtils.GetDurationInfo(playlist);
Console.WriteLine($"Playlist: {info.Count} tracks, total={info.Total},
avg={info.AverageSeconds():0.0}s");
```

Úloha 2 – Hudobná knižnica (9)

- Vytvorte kópiu skladby so zmenou dĺžky/žánru pomocou with
 - Použite switch expression nad Genre

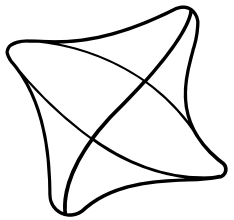
```
var original = Seed.Songs.First();
var longer = original with { Duration = original.Duration + TimeSpan.FromSeconds(30) };

string Badge(Genre g) => g switch
{
    Genre.Classical    => "[CLS]",
    Genre.Jazz         => "[JAZZ]",
    Genre.Rock         => "[ROCK]",
    Genre.Pop          => "[POP]",
    Genre.Electronic   => "[EDM]",
    _                  => "[OTHER]"
};

Console.WriteLine($"{original.Title} -> {Badge(original.Genre)} / {longer.Duration}");
```

Upozornenie

- Tieto študijné materiály sú určené výhradne pre študentov predmetu **Pokročilé objektové technológie** na Fakulte riadenia a informatiky Žilinskej univerzity v Žiline
- Reprodukovanie, šírenie (i častí) materiálov bez písomného súhlasu autora nie je dovolené



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Ing. **Štefan Toth**, PhD.
Katedra softvérových technológií
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
stefan.toth@fri.uniza.sk